

AI Project Leads — Chatbot System Design Document

1. Project Overview

This system extends the existing scraper tool into a full conversational interface. The user interacts with a chatbot, types natural language queries, and gets back structured AI project leads sourced from Freelancer.com, PeoplePerHour, Toptal, and Remotive. All results are stored in MongoDB for caching, history, and trend analysis.

The LLM handles conversation — understanding intent, rephrasing queries, formatting responses, and answering follow-up questions. The scraper handles data — it is the only thing that touches the platforms. These two responsibilities never overlap.

2. Goals

- Replace the terminal workflow with a conversational chat interface
 - Allow natural language queries ("find me RAG projects under \$3000 with low competition")
 - Cache results so repeated queries don't re-scrape
 - Store all historical results in MongoDB
 - Keep the existing scraper engine intact
 - Use Streamlit for the UI
-

4. The LLM's Role — What It Does and Doesn't Do

This is the most important design boundary in the system.

The LLM does:

- Understand the user's natural language query
- Extract the structured search intent from it ("find RAG projects under \$3000" → query: "RAG", filter: budget_max < 3000)
- Decide whether to trigger a fresh scrape or retrieve from cache
- Format raw scraper results into a readable, conversational response
- Answer follow-up questions ("which of those had the lowest competition?", "show me only the Freelancer ones")
- Maintain conversation context across multiple turns
- Handle chitchat and off-topic messages gracefully

The LLM does not:

- Access any website directly
- Call the scraper itself
- Generate or hallucinate project listings
- Make decisions about what to store in the database

The scraper is the single source of truth for all project data. The LLM only ever sees data that has already been scraped and stored.

6. Database Design — MongoDB

Two collections. Nothing more needed at this stage.

Collection 1: `scrape_runs`

Stores metadata about every scrape that was executed.

```
{
  _id:      ObjectId (auto),
  query:    "computer vision",
  normalized_query: "computer vision", ← lowercased, stripped
  ran_at:   ISODate,
  platforms: ["Freelancer.com", "Remotive", "Toptal", "PeoplePerHour"],
  total_results: 24,
  filters_applied: { bid_max: 10 },
  triggered_by: "user_query" ← or "cache_miss"
}
```

Collection 2: `project_listings`

Stores every individual project result. Each document links back to the scrape run that produced it.

```
{
  _id:      ObjectId (auto),
  scrape_run_id: ObjectId, ← foreign key to scrape_runs
  title:    "Object Detection for Warehouse",
  platform: "Freelancer.com",
  description: "...",
  budget_min: 2000,
  budget_max: 4000,
  currency:  "USD",
}
```

```

skills:      ["Python", "OpenCV", "YOLOv8"],
bid_count:   3,
relevance_score: 38.5,
preference_rank: 1,
posted_date: "2026-09-01",
url:         "https://freelancer.com/projects/...",
source_type: "freelance",
keyword:     "computer vision",
first_seen_at: ISODate,      ← when we first scraped this
last_seen_at:  ISODate,      ← updated every time it reappears
times_seen:   1              ← how many scrape runs found this
}

```

Key MongoDB Indexes

These make queries fast as the database grows:

```

scrape_runs: { normalized_query: 1, ran_at: -1 }
project_listings: { scrape_run_id: 1 }
project_listings: { url: 1 }           ← for deduplication
project_listings: { relevance_score: -1 }
project_listings: { platform: 1, first_seen_at: -1 }

```

7. Cache Logic

Cache expiry is set to **4 hours** by default. This is configurable.

If cache is valid → fetch from `project_listings` where `scrape_run_id` matches the most recent run for that query.

If cache is expired or query is new → run scraper → store results → return results.

The user can also force a fresh scrape by saying "search again" or "get fresh results" — the LLM detects this intent and bypasses the cache check.

8. LLM Intent Extraction

When the user sends a message, the LLM doesn't just pass it to the scraper raw. It first extracts a structured intent. This is done with a system prompt and a short JSON response format:

System: You are an assistant that helps find AI development projects.

When the user asks to search for something, extract:

- action: SEARCH | FILTER | RECALL | CHITCHAT
- query: the core search term
- filters: { bid_max, budget_min, budget_max, platform }

Respond only with JSON.

User: "find me RAG pipeline projects under \$5000 on Freelancer"

LLM responds:

```
{  
  "action": "SEARCH",  
  "query": "RAG pipeline",  
  "filters": {  
    "budget_max": 5000,  
    "platform": "Freelancer.com"  
  }  
}
```

Four action types:

Action	What it means	What happens
SEARCH	User wants new results	Check cache → scrape or retrieve
FILTER	User wants to narrow existing results	Re-filter from current session context, no scrape
RECALL	User asks about past results ("what did we find last Tuesday?")	Query MongoDB by date, return stored results
CHITCHAT	Off-topic or greeting	LLM responds conversationally, no scrape

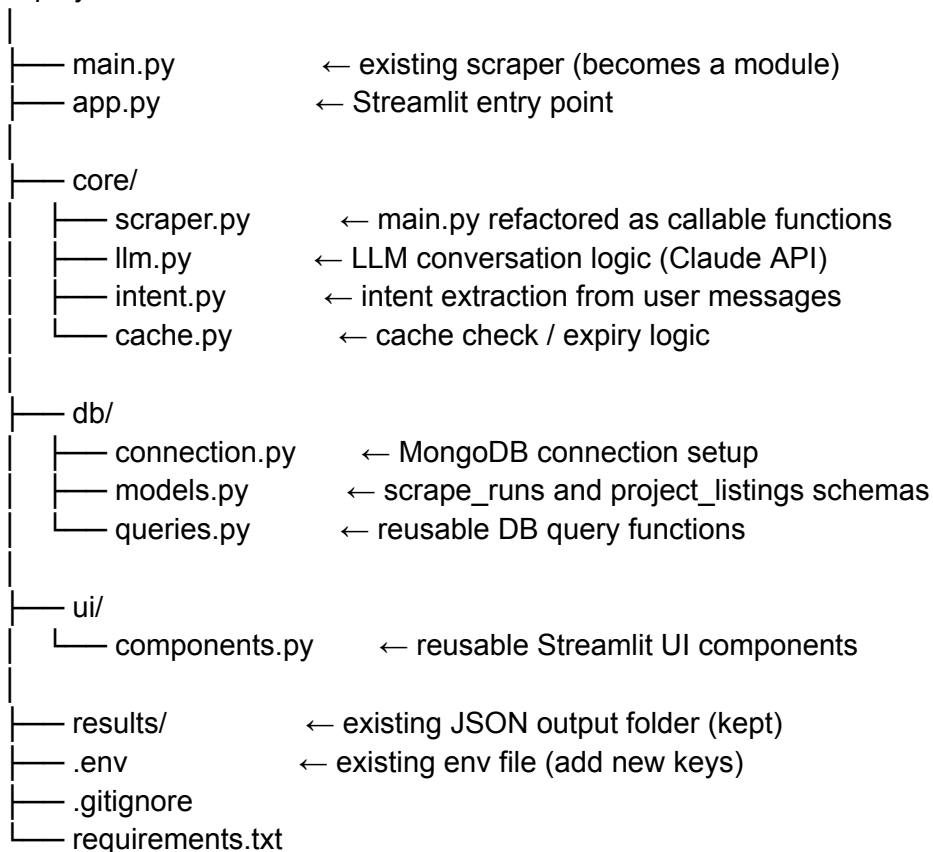
9. Streamlit UI — What It Looks Like

Single page, with some of these components:

- Cache expiry hours (slider)
 - Minimum relevance score (slider)
 - Platforms to include (checkboxes)
 - Force fresh scrape toggle
-

10. Project File Structure

ai-project-leads/



13. Build Order

The timeline is as followed

Phase 1 — Database layer

Set up MongoDB locally. Write the connection module. Write insert and query functions. Test by inserting one of your existing JSON result files manually and querying it back.

Phase 2 — Scraper as a module

Refactor `main.py` so the scraper can be called as a Python function rather than a script. `run(query)` should return a list of `ProjectListing` objects. Test by importing and calling it from a Python shell.

Phase 3 — Cache logic

Write the cache check function. Given a query string, it should return either the cached results from MongoDB or None if the cache is expired. Test with intentional delays.

Phase 4 — LLM intent extraction

Write the intent extractor. Feed it sample user messages and verify it correctly identifies SEARCH vs FILTER vs RECALL vs CHITCHAT and extracts the right query and filters.

Phase 5 — Conversation manager

Wire together: user message → intent extraction → cache check or scrape → LLM formats response → return to UI. Test end to end from Python before touching Streamlit.

Phase 6 — Streamlit UI

Build the chat interface. Connect it to the conversation manager. Add the sidebar settings. Style the project cards.

15. Key Design Principles

- **Scraper and LLM are fully separate.** The LLM never touches a platform. The scraper never touches the LLM.
- **MongoDB is the single source of truth.** The LLM only formats what's in the database. It never generates project data.
- **Cache prevents hammering platforms.** Same query within 4 hours hits the database, not the websites.
- **Streamlit keeps everything in Python.** No JavaScript, no separate frontend, no API endpoints to write.